COL 764: Information Retrieval and Web Search

Semester I, 2026–27

Assignment 4: Agentic IR Tournament

Iterative Retrieval for Multi-hop Question Answering

Released: Saturday, 31 October 2026
Due: Tuesday, 10 November 2026, 23:59 IST (hard deadline; late policy as per Lecture 1)
Weight: 12.5% of the course grade | **Individual assignment**

In one paragraph

You are given a working *single-shot* retrieval-augmented QA system: it runs one BM25 search, passes the top passages to a small language model, and returns an answer. On *two-hop* questions this often fails, because the passage containing the answer can only be found once you know an intermediate ("bridge") entity. Your job is to turn this system into an **iterative retrieval agent** that decides *what to search for next* and *when to stop*, within a fixed budget of search and LLM calls. You write roughly 100–150 lines of Python. Indexing, the language model, and evaluation are all provided.

1 Background and learning objectives

Multi-hop questions such as *"Who is the spouse of the director of the film that won Best Picture in 1998?"* require evidence from more than one document, and the query needed for the second document depends on what was found in the first. IRCoT [2] showed that interleaving retrieval with step-by-step reasoning — using each intermediate reasoning step as the next query — substantially improves both retrieval recall and answer accuracy over one-shot retrieval on such questions. ReAct [3] frames the same idea more generally as an agent alternating between *thought* and *action* (here, a search call).

After completing this assignment you should be able to:

(1) explain why one-shot retrieval fails on bridge-style multi-hop questions;

(2) implement an iterative retrieve–reason loop with explicit query reformulation and a stopping rule;

(3) work within a fixed retrieval and inference budget;

(4) evaluate an end-to-end QA system for both answer quality and evidence quality, and analyse its failures.

2 Data, tools and the language model

2.1 Questions and corpus

- **Questions:** the *2-hop* subset of MuSiQue-Ans v1.0 [1]. MuSiQue was constructed so that its questions cannot be answered by single-hop shortcuts, and every question comes with labelled supporting paragraphs.
- **Corpus:** following IRCoT [2], all supporting and distractor paragraphs of all MuSiQue questions (train, dev and test) are pooled into one corpus of about 139K paragraphs. Each paragraph has a `doc_id`, a `title` and a `text`.
- **Splits:** a public `dev` set of about 300 questions (with gold answers, answer aliases and supporting `doc_ids`) for your own development, and a hidden `test` set of about 300 questions used for the final leaderboard.

2.2 The tool interface

Your agent may interact with the world *only* through the `tools` object passed to it:

```
tools.search(query: str, k: int = 5) -> list[Passage]  # BM25, 1 <= k <= 10
tools.llm(prompt: str, max_tokens: int = 256) -> str    # temperature 0
tools.budget_left() -> dict      # calls remaining for this question

# Passage has fields: doc_id (str), title (str), text (str)
```

BM25 parameters and the index are fixed by staff; you do not build or tune the index in this assignment.

2.3 The language model

All students use the same frozen, staff-hosted instruction-tuned model: **[TBD: model name and size]**, served at **[TBD: endpoint URL]**. The model and decoding settings are identical for everyone, so differences between submissions come from *your retrieval strategy*, not from model choice. You may not call any other model or external service.

2.4 Budget (per question)

The harness enforces the following limits on *every* question:

Resource	Limit per question
`tools.search` calls	5
`tools.llm` calls	6
Prompt length per LLM call	3,000 tokens (longer prompts raise an error and are not counted)
Completion length per LLM call	256 tokens
Wall-clock time	60 seconds

Once a limit is reached, further calls of that kind raise `BudgetExhausted`. Your agent should catch this and still return its best answer. These limits are **[TBD: provisional; final values confirmed at release]**.

3 What you implement

The starter repository contains `agent.py` with a fixed entry point and a working baseline:

```
def run(question: str, tools) -> dict:
    """Must return {"answer": str, "citations": list[str]}  (at most 3 doc_ids)."""
```

The provided `run` implements the single-shot baseline **B1**: one search with the question as query, top-5 passages, one LLM call to answer. A skeleton of the iterative loop is also provided:

```
def run(question, tools):
    notes = Notes()       # passages seen so far + short reasoning steps
    query = question
    while True:
        notes.add(tools.search(query, k=5))
        if should_stop(question, notes, tools):
            break
        query = next_query(question, notes, tools)
    return answer(question, notes, tools)
```

You must implement the three functions below (marked `TODO` in the starter code):

(a) `next_query(question, notes, tools) -> str`: produce the next search query from the evidence gathered so far. A typical approach asks the LLM to state the missing fact (e.g. the bridge entity found in the first hop) and turns it into a query.

(b) `should_stop(question, notes, tools) -> bool`: decide whether enough evidence has been collected. It must also stop safely when the budget runs out.

(c) `answer(question, notes, tools) -> dict`: produce a short answer (an entity, date or number, not a sentence) and cite at most 3 `doc_ids` that support it.

You may change the loop, add helper functions, and use simple non-LLM techniques (e.g. deduplicating passages, keeping only the top passages in the prompt, extracting titles as candidate bridge entities). You must keep the signature of `run`, and your loop must actually be *iterative*: the second and later queries must depend on what was retrieved earlier.

4 Evaluation and leaderboard

4.1 Metrics

For each question, answers are normalised as in SQuAD [4] (lower-casing; removing punctuation, articles and extra whitespace) and compared against the gold answer and all of its aliases, taking the best match.

- **Answer F1** (F_{ans}): token-level F1 between predicted and gold answer. Exact match is also reported but is not used for grading.
- **Evidence F1** (F_{ev}): F1 between your cited `doc_ids` and the gold supporting paragraphs of the question. If you cite more than 3 `doc_ids`, only the first 3 are used.

A crash, a timeout or malformed output on a question scores 0 on that question. The leaderboard score is

$$S = 0.8\overline{F_{\text{ans}}} + 0.2\overline{F_{\text{ev}}},$$

averaged over all questions in the split.

4.2 Leaderboard

- A **live leaderboard** on the public dev set opens on release day at **[TBD: leaderboard URL]**. You may submit at most **3 times per day**; each submission runs in the official harness.
- The **final ranking** uses the hidden test set and is computed once, after the deadline, by staff running your last submitted `agent.py` in the trusted harness.
- Two staff baselines are reported: **B1** (the provided single-shot baseline) and **B2** (an undisclosed staff iterative agent).

5 Grading

Component	What is assessed	Weight
Functionality	Agent runs in the harness on the hidden test set without errors, respects the budget, returns well-formed output, and implements a genuinely iterative loop	20%
Leaderboard	Percentile rank of S on the hidden test set (rule below)	45%
Report	Comparison and error analysis (Section 6)	30%
Code quality	Readable, commented code; no dead or unexplained code	5%

Leaderboard rule. Let S_{B1} be the score of baseline B1 on the hidden test set, and call a submission *eligible* if $S \geq S_{B1}$.

- *Not eligible* ($S < S_{B1}$): you receive $0.4 \times \min\left(1, S/S_{B1}\right)$ of the leaderboard marks.
- *Eligible*: you receive $0.5 + 0.5\,p$ of the leaderboard marks, where $p \in [0,1]$ is your percentile among the N eligible submissions:

$$p \;=\; \frac{\#\{j : S_j < S\} \;+\; \frac{1}{2}\#\{j \neq \text{you} : S_j = S\}}{N-1}$$

  (with $p = 1$ if $N = 1$). Scores are compared to 4 decimal places; tied submissions share the same (mid-rank) percentile.

In other words, simply beating the single-shot baseline guarantees at least half the leaderboard marks.

Viva. Staff may call any student for a short viva. If you cannot explain your own code, your leaderboard marks are capped at 50% of the component.

6 Report (at most 2 pages, PDF)

1. **Single-shot vs. iterative (required table).** On the dev set, report for B1 and for your agent: exact match, answer F1, evidence F1, mean number of search calls, and mean number of LLM calls per question.
2. **Design.** One or two paragraphs explaining your `next_query`, `should_stop` and `answer` choices, and one thing you tried that did *not* help.
3. **Error analysis.** Pick 10 dev questions your agent gets wrong. Label each with one category: (i) *bridge entity not found*, (ii) *stopped too early*, (iii) *evidence retrieved but answered wrongly*, (iv) *answer correct but marked wrong* (e.g. format or alias mismatch), (v) *other* (explain). Give the counts and briefly discuss two examples.
4. **AI-use disclosure and code provenance**, as required in earlier assignments.

7 Submission

Submit a single `<entry_number>.zip` on Moodle containing:

```
agent.py          # your agent (allowed imports: see the starter README)
helpers/          # optional: additional .py files imported by agent.py
report.pdf        # at most 2 pages
```

The last version submitted to the leaderboard must be identical to the `agent.py` in the zip.

8 Rules

- **Individual work.** You may discuss ideas; code and report must be your own.
- **No outside resources at run time.** The harness has no network access other than the staff LLM endpoint. Calling any other model or service is not permitted.
- **No answer lookup.** MuSiQue is public. Submissions may not contain answers, question–answer tables or any data files; the zip must contain only `.py` files and the report, and the code must be under 100 KB. Any attempt to recognise test questions and return stored answers gets zero for the assignment and is treated as academic misconduct.

- **Allowed imports.** A subset of the Python standard library (listed in the starter kit's README), `numpy` and the provided `a4` package. No file access, `os`/`sys`/`subprocess`, `eval`/`exec`. Your agent runs in a sandboxed process; `check_submission.py` applies the same checks as the official harness.
- **Do not tamper with the harness** or try to bypass the budget.

Getting started (suggested plan)

Day 1: Run the B1 baseline on dev and read 20 of its failures. Most will be bridge-entity failures.

Days 2–4: Implement a 2-step loop: search the question, ask the LLM for the bridge entity, search again.

Days 5–7: Add a stopping rule, better answer extraction (short answers!) and citation selection.

Days 8–10: Error analysis, report, final submission.

References

[1] H. Trivedi, N. Balasubramanian, T. Khot, A. Sabharwal. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the ACL (TACL)*, 10:539–554, 2022.

[2] H. Trivedi, N. Balasubramanian, T. Khot, A. Sabharwal. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of ACL 2023*, pages 10014–10037.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR 2023*.

[4] P. Rajpurkar, J. Zhang, K. Lopyrev, P. Liang. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of EMNLP 2016*, pages 2383–2392.